

```

/* GameWin serveur v2 – comptes, codes
d'invitation, mises, dépôts/retraits
validés par l'admin. Montants en F
(solde virtuel tant que le légal n'est pas
réglé). */
const express = require("express");
const cors = require("cors");
const bcrypt = require("bcryptjs");
const jwt = require("jsonwebtoken");
const fs = require("fs");
const crypto = require("crypto");

/* ----- Configuration ----- */
const SECRET = process.env.JWT_SECRET;
const ADMIN_KEY = process.env.ADMIN_KEY;
if (!SECRET || SECRET.length < 16) {
  console.error("JWT_SECRET manquant (16
caractères minimum)"); process.exit(1); }
if (!ADMIN_KEY || ADMIN_KEY.length < 12) {
  console.error("ADMIN_KEY manquant (12
caractères minimum)"); process.exit(1); }
const SITE_URL = process.env.SITE_URL ||
"*";
const FILE = process.env.DATA_FILE ||
"data.json";
const PAY_NUMBER = process.env.PAY_NUMBER
|| "64591190";
const BET_MIN = 220, BET_MAX = 5000;
const DEP_MIN = 1000, DEP_MAX = 10000;
const WD_MIN =

```

```

Number(process.env.WITHDRAW_MIN || 1000),
WD_MAX = Number(process.env.WITHDRAW_MAX ||
10000);
const BONUS_PCT =
Number(process.env.BONUS_PCT ?? 10);
// bonus fidélité (% de la mise)
const START_BONUS =
Number(process.env.START_BONUS || 0);
// solde offert à l'inscription
const REFERRAL_BONUS =
Number(process.env.REFERRAL_BONUS || 0); //
offert au parrain
// À mettre sur "false" dès que de l'argent
réel entre en jeu :
const ALLOW_CLIENT_RESULT =
(process.env.ALLOW_CLIENT_RESULT || "true")
=== "true";

/* ----- Base de données (fichier
JSON) ----- */
function load() {
  const empty = { users: {}, bets: {}, tx:
  {}, ledger: [] };
  if (!fs.existsSync(FILE)) return empty;
  const d =
JSON.parse(fs.readFileSync(FILE, "utf8"));
  return { users:
Object.assign(Object.create(null),
d.users), bets:
Object.assign(Object.create(null), d.bets),
tx:
Object.assign(Object.create(null), d.tx),
ledger: d.ledger || [] };
}

```

```

const db = load();
function save() { fs.writeFileSync(FILE +
".tmp", JSON.stringify(db));
fs.renameSync(FILE + ".tmp", FILE); }

// Tout mouvement de solde passe par ici et
est journalisé
function move(u, delta, reason, ref) {
  if (!Number.isInteger(delta)) throw new
Error("Montant invalide");
  if (u.balance + delta < 0) throw new
Error("Solde insuffisant");
  u.balance += delta;
  db.ledger.push({ t: Date.now(), user:
u.pseudo, delta, reason, ref: ref || null,
solde: u.balance });
}
const isAmount = (n, a, b) =>
Number.isInteger(n) && n >= a && n <= b;
const key = p => String(p ||
 "").trim().toLowerCase();

function newInviteCode() {
  for (;;) {
    const c =
crypto.randomBytes(4).toString("hex").slice
(0, 6).toUpperCase();
    if (!Object.values(db.users).some(u =>
u.invite === c)) return c;
  }
}

/* ----- Application ----- */
const app = express();

```

```

app.set("trust proxy", 1); // Render :
vraie IP du joueur
app.use(express.json({ limit: "10kb" }));
app.use(cors({ origin: SITE_URL,
allowedHeaders: ["Content-Type",
"Authorization", "x-admin-key"] }));

const hits = {};
function limiter(max) {
  return (req, res, next) => {
    const k = req.ip + req.path.split("/")
[1] + max, m = Math.floor(Date.now() /
60000);
    hits[k] = hits[k] && hits[k].m === m ?
hits[k] : { m, n: 0 };
    if (++hits[k].n > max) return
res.status(429).json({ error: "Trop de
requêtes, réessaie dans une minute" });
    next();
  };
}
setInterval(() => { const m =
Math.floor(Date.now() / 60000); for (const
k in hits) if (hits[k].m < m) delete
hits[k]; }, 120000);
app.use(limiter(120));

function auth(req, res, next) {
  try {
    const t = (req.headers.authorization ||
"").replace("Bearer ", "");
    const u = db.users[jwt.verify(t,
SECRET).k];
    if (!u) throw 0;
  }
}

```

```

        if (u.banned) return
        res.status(403).json({ error: "Compte
suspendu" });
        req.user = u; next();
    } catch (e) { res.status(401).json({
error: "Connecte-toi" }); }
}
function admin(req, res, next) {
    const a =
Buffer.from(String(req.headers["x-admin-
key"] || "")), b = Buffer.from(ADMIN_KEY);
    if (a.length !== b.length ||
!crypto.timingSafeEqual(a, b)) return
    res.status(401).json({ error: "Accès admin
refusé" });
    next();
}
const token = u => jwt.sign({ k:
key(u.pseudo) }, SECRET, { expiresIn: "7d"
});
const publicUser = u => ({ pseudo:
u.pseudo, balance: u.balance, invite:
u.invite, referrals: u.referrals || 0 });

/* ----- Compte ----- */
app.get("/", (req, res) =>
res.send("GameWin serveur OK"));

app.post("/register", limiter(10), async
(req, res) => {
    const { pseudo, password, age18, invite }
= req.body || {};
    if (typeof pseudo !== "string" || !/^[A-
Za-z0-9_]{3,20}$/.test(pseudo)) return

```

```

res.status(400).json({ error: "Pseudo : 3 à
20 lettres, chiffres ou _" });
    if (typeof password !== "string" ||
password.length < 6 || password.length >
72) return res.status(400).json({ error:
"Mot de passe : 6 caractères minimum" });
    if (age18 !== true) return
res.status(400).json({ error: "Réservé aux
18 ans et plus" });
    let parent = null;
    if (invite) {
        parent = Object.values(db.users).find(u
=> u.invite ===
String(invite).trim().toUpperCase());
        if (!parent) return
res.status(400).json({ error: "Code
d'invitation inconnu" });
    }
    if (db.users[key(pseudo)]) return
res.status(409).json({ error: "Pseudo déjà
pris" });
    const hash = await bcrypt.hash(password,
10);
    if (db.users[key(pseudo)]) return
res.status(409).json({ error: "Pseudo déjà
pris" }); // re-test après l'attente
    const u = db.users[key(pseudo)] = {
pseudo, hash, balance: 0, invite:
newInviteCode(), referredBy: parent ?
parent.pseudo : null,
        referrals: 0, activeBet: null, banned:
false, created: Date.now() };
    if (START_BONUS) move(u, START_BONUS,
"bonus inscription");

```

```

    if (parent) { parent.referrals++; if
(REFERRAL_BONUS) move(parent,
REFERRAL_BONUS, "parrainage", u.pseudo); }
    save();
    res.json({ token: token(u),
...publicUser(u) });
});

app.post("/login", limiter(10), async (req,
res) => {
    const { pseudo, password } = req.body ||
{};
    const u = db.users[key(pseudo)];
    const ok = u && typeof password ===
"string" && await bcrypt.compare(password,
u.hash);
    if (!ok) return res.status(401).json({
error: "Pseudo ou mot de passe incorrect"
});
    if (u.banned) return
res.status(403).json({ error: "Compte
suspendu" });
    res.json({ token: token(u),
...publicUser(u) });
});

app.get("/me", auth, (req, res) =>
res.json(publicUser(req.user)));

app.get("/classement", (req, res) => {
    res.json(Object.values(db.users).filter(u
=> !u.banned).sort((a, b) => b.balance -
a.balance).slice(0, 20)
    .map((u, i) => ({ rang: i + 1, pseudo:

```

```

u.pseudo, balance: u.balance }));
});

app.get("/transactions", auth, (req, res)
=> {
  res.json(Object.values(db.tx).filter(t =>
t.user === req.user.pseudo).sort((a, b) =>
b.created - a.created).slice(0, 50)
    .map(({ id, type, amount, status,
created, ref, phone }) => ({ id, type,
amount, status, created, ref, phone })));
});

/* ----- Dépôts ----- */
// 1) Le joueur choisit un montant, reçoit
le code USSD à composer
app.post("/deposit/request", auth,
limiter(10), (req, res) => {
  const { amount, operator } = req.body ||
{};
  if (!isAmount(amount, DEP_MIN, DEP_MAX))
return res.status(400).json({ error: `Dépôt
: ${DEP_MIN} à ${DEP_MAX} F` });
  const mine =
Object.values(db.tx).filter(t => t.user ===
req.user.pseudo && t.type === "deposit" &&
t.status === "pending");
  if (mine.length >= 3) return
res.status(409).json({ error: "Tu as déjà 3
dépôts en attente" });
  const id = crypto.randomUUID();
  db.tx[id] = { id, user: req.user.pseudo,
type: "deposit", amount, operator: operator
=== "moov" ? "moov" : "orange", status:

```

```

"pending", created: Date.now(), ref: null
};
  save();
  res.json({ id, amount, ussd:
`*144*10*${PAY_NUMBER}*${amount}#`,
    message: "Compose ce code, puis envoie
l'ID de transaction reçu par SMS à l'étape
suivante." });
});
// 2) Le joueur donne l'ID de transaction
de son SMS (unique : un SMS ne sert qu'une
fois)
app.post("/deposit/confirm", auth,
limiter(10), (req, res) => {
  const { id, ref } = req.body || {};
  const t = db.tx[id];
  if (!t || t.user !== req.user.pseudo ||
t.type !== "deposit" || t.status !==
"pending") return res.status(404).json({
error: "Dépôt introuvable" });
  const r = String(ref ||
"".trim().toUpperCase());
  if (! /^[A-Z0-9.\-]{6,40}$/ .test(r))
return res.status(400).json({ error: "ID de
transaction invalide" });
  if (Object.values(db.tx).some(x => x.type
=== "deposit" && x.ref === r)) return
res.status(409).json({ error: "Cet ID de
transaction a déjà été utilisé" });
  t.ref = r; save();
  res.json({ ok: true, message: "Reçu. Ton
solde sera crédité dès que le paiement est
confirmé." });
});

```

```

/* ----- Retraits ----- */
app.post("/withdraw/request", auth,
limiter(10), (req, res) => {
  const { amount, phone } = req.body || {};
  if (!isAmount(amount, WD_MIN, WD_MAX))
return res.status(400).json({ error:
`Retrait : ${WD_MIN} à ${WD_MAX} F` });
  if (!/^\+?[0-9]{8,15}$/.test(String(phone
|| ""))) return res.status(400).json({
error: "Numéro invalide" });
  if (req.user.activeBet) return
res.status(409).json({ error: "Termine ta
partie en cours d'abord" });
  const id = crypto.randomUUID();
  try { move(req.user, -amount, "retrait
demandé", id); } catch (e) { return
res.status(400).json({ error: e.message });
}
  db.tx[id] = { id, user: req.user.pseudo,
type: "withdraw", amount, phone, status:
"pending", created: Date.now() };
  save();
  res.json({ id, balance: req.user.balance,
message: "Demande envoyée. Le montant est
réservé jusqu'à validation." });
});

/* ----- Mises ----- */
app.post("/miser", auth, (req, res) => {
  const { stake, game } = req.body || {};
  const u = req.user;
  if (!isAmount(stake, BET_MIN, BET_MAX))
return res.status(400).json({ error: `Mise

```

```

: ${BET_MIN} à ${BET_MAX} F` });
  if (u.activeBet) return
res.status(409).json({ error: "Une partie
est déjà en cours" });
  const id = crypto.randomUUID();
  try { move(u, -stake, "mise", id); }
catch (e) { return res.status(400).json({
error: e.message }); }
  db.bets[id] = { id, user: u.pseudo,
stake, game: String(game ||
"autre").slice(0, 20), done: false,
created: Date.now() };
  u.activeBet = id; save();
  res.json({ betId: id, balance: u.balance
});
});

// Règlement unique : victoire ×2, nul
remboursé, défaite perdue, + bonus fidélité
function settle(u, b, outcome) {
  if (b.done) throw new Error("Pari déjà
régulé");
  b.done = true; u.activeBet = null;
b.outcome = outcome;
  const bonus = BONUS_PCT ? Math.max(1,
Math.round(b.stake * BONUS_PCT / 100)) : 0;
  const back = outcome === "win" ? b.stake
* 2 : outcome === "draw" ? b.stake : 0;
  if (back) move(u, back, outcome === "win"
? "mise gagnée" : "mise rendue", b.id);
  if (bonus) move(u, bonus, "bonus
fidélité", b.id);
  return { outcome, back, bonus, balance:
u.balance };
}

```

```

}
app.post("/resultat", auth, (req, res) => {
  if (!ALLOW_CLIENT_RESULT) return
  res.status(403).json({ error: "Ce jeu n'est
pas encore validé côté serveur" });
  const { betId, outcome } = req.body ||
{};
  const b = db.bets[betId], u = req.user;
  if (!b || b.user !== u.pseudo || b.done)
return res.status(404).json({ error: "Pari
introuvable ou déjà réglé" });
  if (b.game === "number") return
res.status(400).json({ error: "Ce jeu se
règle sur le serveur" });
  if (!["win", "lose",
"draw"].includes(outcome)) return
res.status(400).json({ error: "Issue
invalide" });
  const r = settle(u, b, outcome); save();
res.json(r);
});

/* ----- Devine le nombre : 100 % côté
serveur ----- */
const LEVELS = [{ max: 100, tries: 8 }, {
max: 500, tries: 9 }, { max: 1000, tries:
10 }];
app.post("/number/start", auth, (req, res)
=> {
  const { stake, level = 0 } = req.body ||
{};
  const u = req.user, L = LEVELS[level];
  if (!L) return res.status(400).json({
error: "Niveau invalide" });

```

```

    if (!isAmount(stake, BET_MIN, BET_MAX))
    return res.status(400).json({ error: `Mise
: ${BET_MIN} à ${BET_MAX} F` });
    if (u.activeBet) return
res.status(409).json({ error: "Une partie
est déjà en cours" });
    const id = crypto.randomUUID();
    try { move(u, -stake, "mise", id); }
    catch (e) { return res.status(400).json({
error: e.message }); }
    db.bets[id] = { id, user: u.pseudo,
stake, game: "number", secret: 1 +
crypto.randomInt(L.max), left: L.tries,
max: L.max, done: false, created:
Date.now() };
    u.activeBet = id; save();
    res.json({ betId: id, max: L.max, tries:
L.tries, balance: u.balance });
  });
  app.post("/number/guess", auth, (req, res)
=> {
    const { betId, guess } = req.body || {};
    const b = db.bets[betId], u = req.user;
    if (!b || b.user !== u.pseudo || b.done
|| b.game !== "number") return
res.status(404).json({ error: "Partie
introuvable" });
    if (!Number.isInteger(guess) || guess < 1
|| guess > b.max) return
res.status(400).json({ error: "Nombre
invalide" });
    b.left--;
    const won = guess === b.secret;
    if (won || b.left <= 0) { const r =

```

```

settle(u, b, won ? "win" : "lose"); save();
return res.json({ finished: true, won,
secret: b.secret, ...r }); }

    save();
    res.json({ finished: false, hint: guess <
b.secret ? "plus grand" : "plus petit",
left: b.left });
});

/* ----- Admin (en-tête x-admin-key) -
----- */
app.get("/admin/pending", admin, (req, res)
=> {
    res.json(Object.values(db.tx).filter(t =>
t.status === "pending").sort((a, b) =>
a.created - b.created));
});
app.post("/admin/tx/:id/:action", admin,
(req, res) => {
    const t = db.tx[req.params.id], act =
req.params.action;
    if (!t || t.status !== "pending") return
res.status(404).json({ error: "Transaction
introuvable ou déjà traitée" });
    if (!["approve", "reject"].includes(act))
return res.status(400).json({ error:
"Action invalide" });
    const u = db.users[key(t.user)];
    if (t.type === "deposit") {
        if (act === "approve") { if (!t.ref)
return res.status(400).json({ error: "Le
joueur n'a pas encore donné son ID de
transaction" }); move(u, t.amount, "dépôt
confirmé", t.id); }

```

```
    } else if (act === "reject") move(u,
t.amount, "retrait refusé (remboursé)",
t.id);
    t.status = act === "approve" ? "approved"
: "rejected"; t.closed = Date.now();
save();
    res.json({ ok: true, status: t.status });
});
app.post("/admin/ban/:pseudo", admin, (req,
res) => {
    const u =
db.users[key(req.params.pseudo)];
    if (!u) return res.status(404).json({
error: "Inconnu" });
    u.banned = req.body && req.body.banned
!= false; save(); res.json({ pseudo:
u.pseudo, banned: u.banned });
});

app.listen(process.env.PORT || 3000, () =>
console.log("GameWin serveur prêt"));
```